



PhysicalRisk Platform

Developer Guide

MKM Research Labs

Version 1.0 — 14-February-2026

David K Kelly

CONFIDENTIAL — PROPRIETARY

Legal Notice

PROPRIETARY AND CONFIDENTIAL

This document, including all algorithms, models, methodology, formulas, calculations, and intellectual concepts contained herein, constitutes the exclusive intellectual property and confidential trade secrets of MKM Research Labs (“MKM”).

Any usage, reproduction, distribution, modification, or disclosure of this document or any portion thereof, without the express prior written authorisation from MKM Research Labs is strictly prohibited and constitutes an infringement of intellectual property rights.

The document is provided on an “as is” basis. MKM Research Labs makes no representations or warranties of any kind, express or implied, regarding its accuracy, reliability, or suitability for any particular purpose.

All rights reserved. © 2021–2026 MKM Research Labs.

Governed by the laws of the United Kingdom.

Contents

1	Executive Summary	5
1.1	Capabilities	5
1.2	Architecture Overview	5
2	Installation and Setup	6
2.1	Prerequisites	6
2.2	Installation	6
2.3	Environment Variables	6
3	Configuration	6
3.1	Configuration System	6
3.2	Directory Structure	7
3.3	Catchment System	7
4	CLI Reference	8
4.1	server — Start Web Server	8
4.2	port — Generate Portfolio Data	8
4.2.1	Segment Flags	8
4.2.2	Global Options	9
4.3	visual — Generate Interactive Map	9
4.4	check — Verify Dependencies	9
4.5	config — Show Configuration	9
5	Data Pipeline	9
5.1	Generation Sequence	9
5.2	Data Backup	10
5.3	Segment Details	10
5.3.1	Gauges	10
5.3.2	Properties	10
5.3.3	Mortgages	10
5.3.4	Gauge Time Series (gaugets)	10
5.3.5	Storms	10
5.3.6	Gauge Historical Data (gaugehd)	11
5.3.7	Hazard Curves	11
5.3.8	Property Time Series (propertyts)	11
5.3.9	Property Hazard Curves (propertyhc)	11
5.3.10	Counterparties	11
5.4	Common Data Model (CDM)	11
6	Web Server and API Reference	12
6.1	Starting the Server	12
6.2	Blueprint Organisation	12

6.3	Response Format	12
6.4	Health and System Endpoints	13
6.5	Property Endpoints	13
6.6	Gauge Endpoints	13
6.7	Property Flood Timeseries Endpoints	14
6.8	Property Hazard Curve Endpoints	14
6.9	Counterparty and PRS Endpoints	14
7	Interactive Visualisation	14
7.1	Map Generation	14
7.2	Map Layers	14
7.2.1	Gauge Layer	14
7.2.2	Property Layer	15
7.2.3	Mortgage Layer	15
7.3	Context Menus	15
7.4	Interactive Panels	15
7.4.1	Property Storm Analysis	15
7.4.2	Gauge Storm Analysis	16
7.4.3	Property Hazard Curve and PRS Pricing	16
7.4.4	Gauge Hazard Curve	16
7.4.5	Gauge Historical Data	16
7.4.6	Flood Animation	16
7.4.7	Mortgage Detail Panel	17
7.4.8	PDF Report Viewers	17
7.5	Notification System	17
7.6	Backend Communication	17
8	PDF Reports	17
8.1	Property Reports	17
8.2	Gauge Reports	18
8.3	Risk Portfolio Reports	18
8.4	PRS Trade Confirmations	18
9	Analytical Models Reference	18
9.1	Hazard Models	19
9.2	Flood Risk Models	19
9.3	Valuation and Risk Models	19
9.4	Statistics and Intensity Models	19
9.5	Model Documentation	20
10	Docker Deployment	20
10.1	Container Image	20
10.2	Docker Compose	20

11 Testing	20
11.1 Test Architecture	20
11.2 Running Tests	20
11.3 Test Conventions	21
Appendix A: Data File Reference	22
12 Document History	22

1 Executive Summary

PhysicalRisk is a flood risk assessment and Physical Risk Swap (PRS) pricing platform developed by MKM Research Labs. The platform generates synthetic portfolios of flood gauges, residential properties, and mortgages within a river catchment, then simulates storm events to compute property-level flood hazard curves, risk-adjusted mortgage valuations, and PRS contract pricing.

1.1 Capabilities

- **Synthetic portfolio generation** — configurable portfolios of gauges, properties, mortgages, storms, and counterparties.
- **Flood simulation** — storm-to-gauge response modelling with spatial interpolation to property level using inverse distance weighting.
- **Hazard curve construction** — GEV-fitted exceedance probabilities at gauge and property level.
- **PRS pricing** — CDS-equivalent quarterly pricing of Physical Risk Swaps with five-component basis waterfall analysis.
- **Mortgage pricing** — risk-adjusted mortgage valuation with affordability, LTV, and flood risk credit spread adjustments.
- **Interactive visualisation** — Folium-based map with tabbed analysis panels, context menus, flood animation, and PDF report generation.
- **REST API** — Flask server exposing all data and analytics via JSON endpoints.
- **PDF reporting** — automated property, gauge, risk portfolio, and PRS trade confirmation reports.

1.2 Architecture Overview

```

1 app.py (CLI)
2 |-- server      --> src/server.py --> src/routes/*.py (Flask API)
3 |-- port       --> src/port/src/*.py (generators)
4 |             |-- src/port/cdm/*.py (schemas)
5 |             |-- src/port/rand/thames/*.py (random modules)
6 |             |-- src/models/*.py (analytical models)
7 |-- visual     --> src/visual/ (Folium map + interactivity)
8 |-- check      --> dependency verification
9 |-- config     --> configuration display
10
11 config.py      --> PortfolioConfig (singleton, paths, catchment)
12 data/
13 |-- input/thames/ (generated portfolios + timeseries)
14 |-- output/      (PDF reports, PRS trades)
15 |-- results/     (interactive visualisation HTML)

```

Listing 1: Platform architecture

2 Installation and Setup

2.1 Prerequisites

- Python 3.13 or later
- GDAL system library (for geospatial operations)
- QuantLib (optional, for CDS pricing — analytical fallback available)
- Node.js and npm (optional, for JavaScript tests)
- L^AT_EX distribution (optional, for building documentation PDFs)

2.2 Installation

```

1 # Clone repository
2 git clone <repository-url> PhysicalRisk
3 cd PhysicalRisk
4
5 # Install Python dependencies
6 pip install -r requirements.txt
7
8 # Verify installation
9 python app.py check

```

Listing 2: Installation steps

The `check` command verifies all required packages: `flask`, `flask_cors`, `folium`, `pandas`, `numpy`, `geopandas`, `reportlab`, `scipy`, `sklearn`, `rasterio`, `geopy`, `shapely`, `matplotlib`, `seaborn`. Each is reported as OK or MISSING.

2.3 Environment Variables

All settings have sensible defaults but can be overridden:

Table 1: Environment variable configuration

Variable	Default	Description
MKM_SERVER_HOST	127.0.0.1	Server bind address
MKM_SERVER_PORT	5013	Server port
MKM_DEBUG	false	Flask debug mode
MKM_CATCHMENT	thames	Active catchment ID
MKM_PROJECT_ROOT	Auto-detected	Override project root
MKM_INPUT_DIR	data/input	Override input directory
MKM_OUTPUT_DIR	data/output	Override output directory
MKM_RESULTS_DIR	data/results	Override results directory

3 Configuration

3.1 Configuration System

The platform uses two configuration classes in `config.py`:

- **Config** — simple class with server settings as class attributes. Used by Flask for host/port/debug/CORS.
- **PortfolioConfig** — singleton managing catchment, directory paths, and module loading. The global config instance is available via `from config import config`.

```

1 $ python app.py config
2 Project root:    /path/to/PhysicalRisk
3 Catchment:      thames
4 Input dir:      /path/to/PhysicalRisk/data/input/thames
5 Output dir:     /path/to/PhysicalRisk/data/output
6 Reports dir:    /path/to/PhysicalRisk/data/output
7 Results dir:    /path/to/PhysicalRisk/data/results
8 Server URL:     http://127.0.0.1:5013
9 Debug:         False

```

Listing 3: View current configuration

3.2 Directory Structure

```

1 PhysicalRisk/
2   app.py           CLI entry point
3   config.py        Configuration (adds src/ to sys.path)
4   requirements.txt  Python dependencies
5   Dockerfile       Container image
6   docker-compose.yml Container orchestration
7   src/
8     server.py      Flask application factory
9     routes/        API endpoint blueprints
10    visual/         Map visualisation + interactivity
11    models/         Analytical models (centralised)
12    loaders/        Data loading classes
13    port/           Portfolio generators + CDM + random
14    reports/        PDF report generators
15    static/         Web assets (HTML, JS)
16    catch/          Catchment definitions
17    floodts/        Flood timeseries utilities
18  data/
19    input/<catchment>/ Generated portfolio data
20    output/         PDF reports and PRS trades
21    results/        Interactive visualisation HTML
22  tests/           Python and JavaScript tests
23  docs/            Documentation (LaTeX)

```

Listing 4: Project directory layout

3.3 Catchment System

Each catchment defines its geographic region in `data/catch/<id>.py`, inheriting from `BaseCatchment`:

- `CATCHMENT_ID` — unique identifier (e.g. `thames`)
- `GAUGE_POINTS` — list of (lat, lon, elevation_m) tuples
- `AREAS` — region/borough names for labelling

- `CENTER_LAT`, `CENTER_LON` — map centre coordinates
- `generate_synchronized_locations(count)` — produce property locations consistent with catchment geography

The Thames catchment defines 40 gauge points along the river from Richmond to Tilbury (elevations 4–11m) across 30 London boroughs. Additional catchments (Rhine, Mississippi, Danube, etc.) are defined but not yet populated with random modules.

4 CLI Reference

All commands are invoked via `python app.py <command> [options]`.

4.1 server — Start Web Server

```
1 python app.py server [--host HOST] [--port PORT] [--debug]
```

Starts the Flask web server. Registers all API route blueprints and serves the interactive visualisation. Default: `http://127.0.0.1:5013`.

4.2 port — Generate Portfolio Data

```
1 python app.py port [SEGMENTS] [OPTIONS]
```

Generates synthetic portfolio data. Segments must be run in dependency order (see Section 5).

4.2.1 Segment Flags

Table 2: Portfolio generation segment flags

Flag	Short	Output	Description
<code>--gauges</code>	<code>--ga</code>	<code>gauge.json</code>	Flood gauge stations
<code>--properties</code>	<code>--pr</code>	<code>property.json</code>	Property portfolio
<code>--mortgages</code>	<code>--mo</code>	<code>mortgage.json</code>	Mortgage data
<code>--gaugets</code>	<code>--gt</code>	<code>gaugets/*.json</code>	Gauge flood simulations
<code>--storms</code>	<code>--st</code>	<code>storms.json</code>	Storm scenarios
<code>--gaugehd</code>	<code>--hd</code>	<code>gaugehd/*.json</code>	Historical daily data
<code>--hazard</code>	<code>--hz</code>	<code>gaugehc.json</code>	Gauge hazard curves
<code>--propertyts</code>	<code>--pt</code>	<code>propertyts/*.json</code>	Property flood events
<code>--propertyhc</code>	<code>--phc</code>	<code>propertyhc.json</code>	Property hazard + PRS
<code>--counterparties</code>	<code>--ctpy</code>	<code>counterparty.json</code>	Swap counterparties
<code>--all</code>	<code>-a</code>	All above	Run all segments

4.2.2 Global Options

Table 3: Portfolio generation options

Flag	Short	Default	Description
--num-properties	-np	200	Number of properties
--num-gauges	-ng	40	Number of gauges
--num-storms	-ns	10,000	Number of storm scenarios
--simulation-hours		60	Flood simulation duration
--history-years	-hy	50	Historical data years
--tail-weight	-tw	2.0	Storm intensity tail weight
--distribution	-d	gev	Hazard distribution (gev gumbel)
--no-backup		false	Skip automatic data backup
--verbose	-v	false	Verbose output

4.3 visual — Generate Interactive Map

```
1 python app.py visual [--no-browser]
```

Generates an interactive Folium map at `data/results/interactive_visualization.html` and opens it in the default browser (unless `--no-browser` is specified).

4.4 check — Verify Dependencies

```
1 python app.py check
```

Reports the status of each required Python package. Exits with code 1 if any packages are missing.

4.5 config — Show Configuration

```
1 python app.py config
```

Displays current project root, catchment, directory paths, server URL, and debug mode.

5 Data Pipeline

5.1 Generation Sequence

Segments must be generated in dependency order. Each segment reads from the output of earlier segments:

```

1 gauges --> properties --> mortgages
2     |
3 gauges --> gaugets --> storms --> gaugehd --> hazard
4     |                                     |
5     +-----> propertyts -----> propertyhc
6                                     |
7                                counterparties -----+
```

Listing 5: Data pipeline dependency chain

Running `python app.py port --all` executes all segments in the correct order. When regenerating specific segments, ensure upstream dependencies exist.

5.2 Data Backup

Before generating new data, the existing input directory is automatically backed up to `audit/<catchment>_YYYYMM`. This preserves an audit trail of all data changes. Use `--no-backup` to skip.

5.3 Segment Details

5.3.1 Gauges

Generates flood gauge stations with location, elevation, and flood stage thresholds (Alert, Warning, Severe). Uses catchment-defined gauge points. Each gauge includes sensor metadata (type, owner, installation date) and historical statistics.

Output: `gauge.json` containing `flood_gauges[]` array.

5.3.2 Properties

Generates residential properties with location, construction details, and multi-factor valuation. Property value is computed as:

$$V = \text{area} \times \frac{\text{price}}{\text{sqm}} \times f_{\text{location}} \times f_{\text{age}} \times f_{\text{condition}} \times f_{\text{flood}} \times f_{\text{EPC}} \times f_{\text{proximity}}$$

The critical `FloorLevelMeters` field determines whether a property floods — only water above this threshold (relative to ground elevation) counts as internal flooding.

Output: `property.json` containing `properties[]` array.

5.3.3 Mortgages

Generates mortgage instruments linked to properties by `PropertyID`. Includes loan principal, interest rate, term, borrower income, and computes affordability ratio and LTV. Each mortgage carries a `FloodRiskCategory` (Very Low through Very High).

Output: `mortgage.json` containing `mortgages[]` array.

5.3.4 Gauge Time Series (gaugets)

Simulates 60-hour flood events for each gauge. Produces hourly water level readings showing baseline, storm rise, peak, and recession. Also generates storm response records linking storms to gauge-level peak levels.

Output: `gaugets/GAUGE-xxx.json` per gauge, containing `flood_simulation` (hourly readings) and `storm_responses`.

5.3.5 Storms

Generates storm scenarios using a Hybrid Normal-Pareto intensity distribution. Most storms are benign; rare events cause significant gauge responses. Each storm has spatial extent, track, duration, and intensity parameters.

Output: `storms.json` containing `storms[]` array (default 10,000 events, ~6 MB).

5.3.6 Gauge Historical Data (gaugehd)

Generates daily water level records spanning the configured number of years (default 50). Computes percentiles (p5–p99), annual maxima, and flood exceedance counts. Can also parse NRFA GDF CSV format.

Output: `gaugehd/GAUGE-xxx.json` per gauge, containing `daily_observations[]` and `statistics`.

5.3.7 Hazard Curves

Reads storms and gauge time series, simulates storm-to-gauge responses, extracts peak levels, and fits a GEV (Generalised Extreme Value) distribution. Computes exceedance probabilities at return periods (1, 5, 10, 25, 50, 100 years) and prices gauge-level PRS contracts.

Output: `gaugehc.json` containing per-gauge GEV parameters, hazard curve points, return period levels, and PRS term structures.

5.3.8 Property Time Series (propertyts)

For each property, identifies the three nearest gauges and interpolates flood depth using inverse distance weighting ($1/d^2$). Applies elevation differentials and floor level thresholds. Uses Manning's equation for travel time and attenuation. Computes damage ratio via depth-damage curve.

Output: `propertyts/PROP-xxx.json` per property, containing `flood_events[]` with depth, damage ratio, arrival time, and hourly readings. Also `portfolio_flood_summary.json` with aggregate statistics.

5.3.9 Property Hazard Curves (propertyhc)

Fits GEV to property flood depths (requires ≥ 3 events). Computes exceedance probabilities at three depth thresholds ($>0\text{m}$, $>0.5\text{m}$, $>1.0\text{m}$). Prices property-level PRS via analytical CDS model. Computes five-component basis waterfall (distance, elevation, terrain, composition, model uncertainty).

Output: `propertyhc.json` containing per-property GEV parameters, hazard curves, PRS spreads, and basis analysis.

5.3.10 Counterparties

Generates synthetic counterparty entities (banks, insurers, reinsurers) with credit ratings, exposure limits, and collateral requirements. Used for PRS trade management.

Output: `counterparty.json`.

5.4 Common Data Model (CDM)

All entities follow standardised schemas defined in `src/port/cdm/`:

Table 4: CDM schema registry

CDM Class	File	Key Sections
FloodGaugeCDM	gauge.py	Header, Location, FloodStages, SensorDetails
PropertyCDM	property.py	Header, Location, Valuation, Construction
MortgageCDM	mortgage.py	Header, FinancialTerms, Borrower, Risk
StormEventCDM	storm.py	StormHeader, Intensity, Spatial, Timing
StormTimeSeriesCDM	stormts.py	TimeseriesHeader, Readings
PhysicalRiskSwapCDM	prs.py	SwapHeader, Pricing, Basis
CounterpartyCDM	ctpy.py	EntityHeader, Exposure, Collateral

All CDMs inherit from `BaseCDM` and provide `validate(data)`, `create_mapping(data)`, and a `schema` property.

6 Web Server and API Reference

6.1 Starting the Server

```
1 python app.py server --port 5013 --debug
```

The server registers route blueprints and serves both the API and the interactive visualisation.

6.2 Blueprint Organisation

Table 5: API route blueprints

Blueprint	Prefix	Purpose
health_bp	/	Health check, root route
catchment_bp	/	Catchment selector page
visualization_bp	/	Interactive map generation
properties_bp	/api/v1	Property endpoints
gauges_bp	/api/v1	Gauge endpoints
propertyts_bp	/api/v1	Property flood timeseries
propertyhc_bp	/api/v1	Property hazard curves
counterparty_bp	/api/v1	Counterparty data
prs_bp	/api/v1	PRS trade management

6.3 Response Format

All API responses use a consistent JSON envelope:

```
1 // Success
2 {"status": "success", "data": {...}}
3
4 // Error
5 {"status": "error", "message": "Description of error"}
```

Listing 6: Standard API response format

HTTP status codes: 200 (success), 400 (bad request), 404 (not found), 500 (server error), 503 (missing dependencies).

6.4 Health and System Endpoints

Table 6: Health and system endpoints

Method	Path	Description
GET	/ or /health	Basic health status
GET	/health/detailed	Detailed health with file checks
GET	/select-catchment	Catchment selection HTML page
POST	/api/set-catchment	Set active catchment
GET	/visualization	Generate and serve interactive map

6.5 Property Endpoints

Table 7: Property API endpoints

Method	Path	Description
GET	/api/v1/properties	List all properties
GET	/api/v1/properties/<id>	Single property details
GET	/api/v1/properties/<id>/mortgage	Mortgage for property
POST	/api/v1/properties/report	Generate property PDF
POST	/api/v1/properties/mortgage-report	Generate mortgage PDF

6.6 Gauge Endpoints

Table 8: Gauge API endpoints

Method	Path	Description
GET	/api/v1/gauges	List all gauges
GET	/api/v1/gauges/<id>	Single gauge details
GET	/api/v1/gauges/<id>/history	Historical daily data
GET	/api/v1/gauges/<id>/timeseries	Flood simulation readings
GET	/api/v1/gauges/<id>/storms	Storm scenario analysis
GET	/api/v1/gauges/<id>/hazard	Hazard curve data
GET	/api/v1/gauges/<id>/statistics	Reading statistics
POST	/api/v1/gauges/report	Generate gauge PDF
POST	/api/v1/price_prs	Price a PRS contract

6.7 Property Flood Timeseries Endpoints

Table 9: Property flood timeseries endpoints

Method	Path	Description
GET	/api/v1/propertyts/summary	Portfolio flood summary
GET	/api/v1/properties/<id>/floods	Flood events for property
GET	/api/v1/properties/<id>/storms	Storm analysis with readings
GET	/api/v1/propertyts/storms	Storms causing property floods
GET	/api/v1/propertyts/animate/<storm_id>	Animation frames (60 hours)
GET	/api/v1/propertyts/animate/composite	Worst-case composite animation

6.8 Property Hazard Curve Endpoints

Table 10: Property hazard curve endpoints

Method	Path	Description
GET	/api/v1/propertyhc/summary	Portfolio hazard summary
GET	/api/v1/properties/<id>/hazard	Property hazard + PRS pricing
GET	/api/v1/propertyhc/basis	Basis analysis table

6.9 Counterparty and PRS Endpoints

Table 11: Counterparty and PRS trade endpoints

Method	Path	Description
GET	/api/v1/counterparties	List counterparties
GET	/api/v1/counterparties/<id>	Counterparty details
POST	/api/v1/prs/commit	Commit a PRS trade
GET	/api/v1/prs/trades	List committed trades
GET	/api/v1/prs/trades/<swap_id>/pdf	Download trade PDF

7 Interactive Visualisation

7.1 Map Generation

```
1 python app.py visual
```

Generates an interactive Folium HTML map with all portfolio data overlaid. The map includes tile layer controls, measurement tools, fullscreen toggle, and a scale indicator.

7.2 Map Layers

7.2.1 Gauge Layer

Gauge markers use water-droplet icons colour-coded by operational status (green = operational, orange = maintenance, red = offline). Tooltips show gauge ID and flood frequency. Click popups display

equipment details, sensor data, flood thresholds (alert/warning/severe levels), and historical context. A RAG indicator shows flood frequency risk.

7.2.2 Property Layer

Property markers use home icons colour-coded by flood frequency from simulated storms (red = high, orange = medium, green = low). Mortgaged properties show a bank icon variant. Click popups display property details, elevation and flood zone information, and mortgage details where applicable.

7.2.3 Mortgage Layer

Optional overlay showing translucent circles around mortgaged properties. Circle size correlates with loan amount; colour indicates flood risk level. High-leverage properties ($LTV > 80\%$) show a filled indicator.

7.3 Context Menus

Right-clicking a marker opens a context menu with actions:

Property context menu:

1. Property Details — view full property information
2. Mortgage Details — open mortgage detail panel
3. View Storm Scenarios — open Property Storm Analysis panel
4. Physical Risk Swap — open hazard curve and PRS pricing panel
5. Generate Property Report — create and view PDF report
6. Generate Mortgage Report — create mortgage-focused PDF

Gauge context menu:

1. View History — show historical water level chart
2. View Storm Scenarios — open Gauge Storm Analysis panel
3. Physical Risk Swap — open gauge hazard curve and PRS pricing
4. Generate Gauge Report — create and view PDF report

7.4 Interactive Panels

7.4.1 Property Storm Analysis

Five-tab panel (780×560px) loaded from `/api/v1/properties/{id}/storms`:

1. **Distribution** — flood depth histogram (0.2m bins) with event count, mean/max depth, gauge-to-property conversion rate.
2. **Flood Timeline** — 60-hour hydrograph for selected storm showing water surface elevation, property floor threshold, gauge reference lines, and red shading where water exceeds floor level. Storm selector dropdown. Stats: peak above floor, damage ratio, arrival/peak hours, flooded duration.
3. **Worst Storms** — horizontal bar chart of top 20 storms by flood depth, colour-coded by severity. Click a bar to switch to Timeline tab for that storm.
4. **Flood History** — scrollable table of all flood events (storm ID, depth, damage %) alongside a bar chart. Click rows to view timelines.

5. **Mortgage Impact** — stacked bar chart showing retained property value (blue) and damage loss (red) per storm. Outstanding balance shown as dashed horizontal line. Summary: property value, outstanding balance, current LTV, worst loss, worst post-damage LTV, negative equity count.

7.4.2 Gauge Storm Analysis

Three-tab panel (780×560px) loaded from `/api/v1/gauges/{id}/storms`:

1. **Distribution** — storm peak level histogram.
2. **Flood Timeline** — water level hydrograph with flood stage thresholds (alert, warning, severe).
3. **Worst Storms** — top 20 storms by peak level.

7.4.3 Property Hazard Curve and PRS Pricing

Four-tab panel (1100×750px) loaded from `/api/v1/properties/{id}/hazard`:

1. **Hazard Curve** — log-log exceedance curve with 1-in-10, 1-in-50, 1-in-100 year flood markers.
2. **Term Structure** — annual flood probability over 1–30 years.
3. **PRS Pricing** — six-component pricing breakdown with basis waterfall. Shows base rate, depth threshold components (>0m, >0.5m, >1.0m), and contingent pricing.
4. **Basis Analysis** — gauge spread vs property spread comparison. Basis = gauge spread – property spread (positive = gauge more expensive, expected).

7.4.4 Gauge Hazard Curve

Five-tab panel (1100×750px) loaded from `/api/v1/gauges/{id}/hazard`:

1. **Hazard Curve** — exceedance probabilities for gauge.
2. **Annual Exceedance** — annual probability by depth.
3. **Return Period** — return period curves (1-in- X years).
4. **PRS Pricing** — QuantLib CDS pricing for gauge-level PRS.
5. **Flood Probability** — probability at different thresholds.

7.4.5 Gauge Historical Data

Single-chart panel (700×500px) showing historical daily water levels with flood stage thresholds as horizontal reference lines. Loaded from `/api/v1/gauges/{id}/history`.

7.4.6 Flood Animation

60-frame animation panel (860×620px) showing storm progression. Loaded from `/api/v1/propertyts/animate/{`

- Storm selector dropdown for all available storms.
- Play/Pause controls with speed adjustment (1x/2x/5x).
- Scrubber bar to jump to any hour.
- Gauge markers change colour by water level (green → red).
- Expanding translucent circles from flooded gauges.
- Property markers appear when flood front arrives.
- Stats bar updates per frame: hour, gauges flooded, properties flooded.

7.4.7 Mortgage Detail Panel

Single-section panel (600×520px) loaded from `/api/v1/properties/{id}/mortgage`. Displays five sections: Loan Summary, Financial Terms, Current Status, Borrower, and Risk Assessment.

7.4.8 PDF Report Viewers

Embedded iframe panels (850px wide, 90% viewport height) for viewing generated PDF reports. Include download and print buttons.

7.5 Notification System

Toast-style notifications provide user feedback for all actions:

- **Info** (blue) — general information
- **Success** (green) — operation completed
- **Warning** (orange) — caution
- **Error** (red) — operation failed
- **Loading** (grey) — operation in progress

Auto-dismiss after 5 seconds. Maximum 5 visible simultaneously. Configurable position (default: top-right).

7.6 Backend Communication

Frontend panels communicate with the Flask API via the `window._BACKEND_CONFIG` object, which stores the server URL and endpoint mappings. The backend handler (`src/static/js/backend-handler.js`) manages fetch requests with CORS support and 30-second timeout.

8 PDF Reports

Reports are generated via the API (POST endpoints) or via context menu actions in the interactive map. All reports use ReportLab and follow a consistent branded template.

8.1 Property Reports

15-page comprehensive property analysis:

1. Title and overview
2. Location (map, coordinates, flood zone)
3. Property attributes (type, construction, value)
4. Construction details (materials, age, storeys)
5. Risk assessment (flood depth, damage analysis)
6. Financial (valuation factors, insurance)
7. Flood protection and insurance
8. Energy and EPC rating
9. Transaction history
10. Timeline
11. Mortgage overview (three sub-pages: details, costs, regulatory)

12. Current status
13. Risk analysis
14. Borrower profile
15. Data summary

Generated by `PropertyReportGenerator` in `src/reports/property/`.

8.2 Gauge Reports

11-page gauge analysis:

1. Title and overview
2. Sensor details (type, owner, certification)
3. Location (map, coordinates, ground elevation)
4. Measurements (frequency, method, transmission)
5. Flood stages (alert/warning/severe thresholds)
6. Risk assessment (flood probability, frequency)
7. Data summary
8. Flood history (past events, timeline)
9. Hazard curves (exceedance probability charts)
10. PRS pricing (credit swap valuation)
11. Current risk assessment

Generated by `GaugeReportGenerator` in `src/reports/gauge/`.

8.3 Risk Portfolio Reports

7-page portfolio-level analysis:

1. Title page
2. Executive summary (key metrics, highlights)
3. Portfolio overview (property and gauge distribution)
4. Risk analysis (aggregate flood risk, concentration)
5. Mortgage analysis (LTV distribution, vulnerability)
6. Property details (ranked risk list)
7. Appendix (technical details, methodology)

Generated by `RiskReportGenerator` in `src/reports/risk/`.

8.4 PRS Trade Confirmations

Generated when a PRS trade is committed via `POST /api/v1/prs/commit`. Stored in `data/output/prs/` as both JSON (CDM record) and PDF (trade confirmation). Downloadable via `GET /api/v1/prs/trades/<swap_id>`.

9 Analytical Models Reference

All analytical functions are centralised in `src/models/`. Portfolio generators and visualisation components are thin wrappers delegating to these models.

9.1 Hazard Models

Table 12: Hazard model modules

Module	Purpose
hazard/gev.py	GEV fitting, exceedance, return levels, term structure
hazard/response_model.py	Gauge response to storms (spatial decay)
hazard/builder.py	End-to-end hazard curve construction
hazard/pricing.py	QuantLib CDS pricing (optional)
hazard/prs_analytical.py	Analytical PRS pricing, basis waterfall
hazard/io.py	Hazard curve file I/O

9.2 Flood Risk Models

Table 13: Flood risk model modules

Module	Purpose
floodrisk/depth_damage.py	Depth-damage curves (UK-calibrated)
floodrisk/spatial.py	Haversine, IDW, KDTree, correlation
floodrisk/velocity.py	Manning's equation, travel time, attenuation
floodrisk/risk_analytics.py	Monte Carlo VaR/ES, HHI, clustering
floodrisk/flood_risk_model.py	Unified flood risk pipeline

9.3 Valuation and Risk Models

Table 14: Valuation, risk, and pricing modules

Module	Purpose
valuation/property_value.py	Multi-factor property valuation
valuation/insurance.py	Insurance premium calculation
risk/risk_assessor.py	Risk scoring, vulnerability, VaR
mortgage_pricer.py	Amortisation, credit spread, expected loss
prs_with_hazard_curve.py	PRS pricing via QuantLib CDS

9.4 Statistics and Intensity Models

Table 15: Statistics, intensity, and storm-gauge modules

Module	Purpose
statistics/timeseries.py	Percentiles, annual maxima, exceedances
statistics/synthetic.py	Stochastic water level generation
intensity/distribution.py	Hybrid Normal-Pareto storm intensity
stormgauge/forward_model.py	Storm-gauge spatial decay, intensity-to-level

9.5 Model Documentation

Detailed mathematical specifications with sensitivity analysis are available as separate PDFs in `docs/models/`. Each model document follows the MKM bank validation protocol with sections for executive summary, purpose, mathematical framework, inputs, implementation, sensitivity, validation, and limitations.

10 Docker Deployment

10.1 Container Image

The Dockerfile builds a production image based on `python:3.13-slim` with GDAL system libraries:

```
1 # Build image
2 docker build -t physicalrisk .
3
4 # Run container
5 docker run -p 5001:5001 -v ./data:/app/data physicalrisk
```

Listing 7: Docker build and run

The container exposes port 5001 and runs `python app.py server --host 0.0.0.0 --port 5001`.

10.2 Docker Compose

```
1 docker-compose up           # Start service
2 docker-compose up --build   # Rebuild and start
3 docker-compose down         # Stop service
4 docker-compose logs -f      # Stream logs
```

Listing 8: Docker Compose usage

Configuration:

- Port mapping: 5001:5001
- Volume: `./data:/app/data` (persists data outside container)
- Environment: `FLASK_ENV=production, CATCHMENT_ID=thames`
- Restart policy: `unless-stopped`

11 Testing

11.1 Test Architecture

The platform has comprehensive test coverage:

- **518 Python tests** across `tests/` (6 skipped)
- **51 JavaScript tests** in `tests/js/`

11.2 Running Tests

```
1 # All Python tests
2 python -m pytest tests/ -v
3
4 # Specific module
```

```
5 python -m pytest tests/models/test_mortgage_pricer.py -v
6
7 # JavaScript tests (requires Node.js)
8 npx jest
9
10 # With coverage
11 python -m pytest tests/ --cov=src --cov-report=html
```

Listing 9: Test execution

11.3 Test Conventions

- Tests use `tmp_path` fixture for file isolation.
- Random output is tested with range assertions, not exact values.
- `tests/port/conftest.py` provides shared fixtures for random modules, sample locations, and synthetic data.
- `tests/js/` uses Jest for frontend component testing (notifications, backend handler, context menus).
- `tests/reports/test_pdf_generation.py` has 28 tests for PDF report generation.

Appendix A: Data File Reference

File / Directory	Format	Key Fields
gauge.json	JSON array	GaugeID, lat/lon, elevation, flood stages
property.json	JSON array	PropertyID, lat/lon, elevation, floor level, value
mortgage.json	JSON array	MortgageID, PropertyID, loan, rate, LTV, income
storms.json	JSON array	storm_id, intensity, duration, spatial extent
counterparty.json	JSON array	entity ID, type, credit rating, limits
gaugehc.json	JSON array	gauge_id, GEV params, hazard points, PRS spreads
propertyhc.json	JSON array	property_id, GEV params, PRS pricing, basis
gaugets/GAUGE-*.json	Per-gauge	flood_simulation (hourly readings), storm_responses
gaugehd/GAUGE-*.json	Per-gauge	daily_observations, statistics, percentiles
propertyts/PROP-*.json	Per-property	flood_events (depth, damage, readings)
output/property/*.pdf	PDF	Property analysis reports (15 pages)
output/gauge/*.pdf	PDF	Gauge analysis reports (11 pages)
output/prs/*.json	JSON	PRS trade CDM records
output/prs/*.pdf	PDF	PRS trade confirmations
results/*.html	HTML	Interactive Folium visualisation

12 Document History

Date	Version	Description	Author
14-Feb-2026	1.0	Initial user guide	D. Kelly